

разработки программного обеспечения
arkup Language, Cascading Style Sheets

Факультет инфокоммуникационных сетей и систем
Кафедра программной инженерии и вычислительной техники
Резников Б. К., Логвинова Е. А.

Технологии разработки программного обеспечения

S3W03 JS (JAVASCRIPT)

ыы.рф/аЕа

Цель работы

- Получить навыки работы с языком программирования JavaScript.
- Получить навыки работы с DOM.

Теоретические сведения

JS — JavaScript

JavaScript — динамически типизированный язык программирования, синхронный, однопоточный, с возможностью выполнения асинхронных задач. Динамическая типизация означает, что одна и та же переменная в разные моменты времени может содержать значения разных типов. JavaScript не накладывает ограничений на перезапись значения переменной, за исключением случаев, когда переменная не может быть изменена. Переменные в JS задаются с помощью ключевых слов `var`, `let` и `const`.

Свойство	Инструкция		
	<code>var</code>	<code>let</code>	<code>const</code>
1 Изменяемость (mutability)	+	+	–
2 Инициализация (initialization)	+	не обязательная	обязательная
3 Область видимости (scope)	функциональная	блочная	блочная
4 Сторонние эффекты (side effects)	добавляется как свойство в объект window	–	–
5 Многократное создание (redeclaration)	+	–	–
6 Всплытие (hoisting)	+	+ (мертвая зона)	+ (мертвая зона)

Типы данных

Значение в JavaScript всегда относится к данным определённого типа. Например, это может быть строка или число. Типы данных в JS делятся на два типа: примитивные и ссылочные.

- **Примитивные:**

- **String.** Обозначаться строки могут как с помощью 'одинарных кавычек', так и с помощью "двойных". Между этими двумя вариантами нет никакой разницы. Однако есть ещё один вариант обозначения: `обратные кавычки`. Они могут включать подстановки или иными словами `\${вычисляемые выражения}`, а также сохраняют переносы строк.

```
const name = 'Katua'
const age = 23
```

```
const message = `Hello, ${name}! Your age is ${age}.` // Hello, Katua! Your age is 23.
```

```
console.log('11' + '1' - 1) // 110, сначала складываются строки -> '111', затем идет вычитание, при котором строка приводится к числу -> 110
```

- **Number.** Тип данных number включает в себя как целочисленные значения (integer, int), так и значения с плавающей точкой (float, double).
- **Boolean.**

```
console.log(Boolean([])); // true
console.log(Boolean(function () {})); // true
console.log(Boolean(Symbol('Hi'))); // true
console.log(Boolean('')); // false
console.log(Boolean(' ')); // true
console.log(Boolean(1)); // true
console.log(Boolean(0)); // false
```

- **Null.** Данное значение означает «пустоту» или «ничего».
- **Undefined.** Специальный тип данных, содержащий всего одно значение. Это значение отражает то, что значение не задано. Например, созданная, но не проинициализированная переменная по умолчанию содержит значение undefined. В объекте при попытке получить значение по ключу, которого там нет, будет получено значение undefined. Также любая функция по умолчанию возвращает значение undefined.
- **Symbol.** Тип symbol в JavaScript - это уникальный идентификатор. Один Symbol никогда не равен другому.
- **BigInt.** BigInt - это тип данных способный содержать в себе целые числа любых размеров без потери точности. Значения типа bigint можно получить при помощи добавления символа n в конец числового литерала или же при помощи вызова встроенной функции BigInt().

- **Ссылочные:**

- **Object.** Object является ссылочным типом данных. Это означает то, что работа со всеми объектами в JavaScript ведётся по ссылке. Данные в объектах хранятся в виде пар «ключ: значение», где ключ - это строка (или symbol), а значение может являться чем угодно (в том числе другим объектом). По сути всё, что не примитив в JavaScript, - это объект. К ним относятся как сами объекты, так и массивы, и даже функции.

```
const a = {
  name: 'Bob'
```

```
}
```

```
const b = {  
  name: 'Bob'  
}
```

```
console.log(a === b) // false, ссылки разные
```

```
const a = {  
  name: 'Bob'  
}
```

```
const b = a
```

```
console.log(a.name === b.name) // true, в b скопировали ссылку a
```

Приведение типов и сравнение в JS

Случаются моменты, когда нужно преобразовать один тип к другому. Есть два вида преобразований: явное и неявное.

Явное приведение типов: с использованием встроенных функций, например: `String()`, `toString()`, `Number()`, `parseInt()`, `Boolean()`.

Неявное приведение:

- Строку к числу — с помощью добавления в начале строки знака +
- Число к строке — с помощью добавления пустой строки
- Строку к числу — с помощью арифметических операторов или оператором сравнения
- К Boolean значению — с помощью добавления спереди двух восклицательных знаков !!

```
const strNum = '5';  
const str = 'hello world';  
const bool = true;  
  
console.log(5 + ''); // '5'  
console.log('5' + 3); // '53'  
console.log(true + false) // 1  
console.log(+strNum); // 5  
console.log(+str); // NaN  
console.log(!bool); // true  
console.log(!bool); // false  
console.log(!"false" == !"true") // true
```

Сравнить примитивные типы данных в JS можно одним из способов:

Двойное равно (==) — с приведением типов данных.

Тройное равно (===) — без приведения типов данных.

```
const a = 5  
const b = '5'  
  
console.log(a == b) // true  
console.log(a === b) // false
```

Объекты. Копирование, сравнение

Как известно, в JS существует 8 типов данных, из которых 7 – примитивные. Отличие ссылочного типа данных от примитивного заключается в том, что значения сравниваются не “по значению”, а “по ссылке”. Объект хранит в себе ссылку на созданный объект, а не сам объект. Именно поэтому при присваивании одному объекту другого присваивается не само значение объекта а ссылка.

```
const obj = {  
  name: 'Gayan Rosling'  
};  
  
const newObj = obj; // Присвоили объекту ссылку, а не сам объект!  
  
console.log(newObj) // { name: 'Gayan Rosling' }  
  
obj.name = 'Harry Potter'  
  
console.log(newObj) // { name: 'Harry Potter' }
```

В примере мы поменяли изначальный объект obj, но значение объекта newObj тоже изменилось.

Есть несколько способов создания объектов:

- литеральная форма

```
const someObj = {};  
const obj = {  
  name: 'Gayan Rosling',  
};
```

- с помощью ключевого слова new

```
const someObj = new Object();
```

- с помощью встроенных методов Object

```
const pureObject = Object.create(null);  
const emptyObject = Object.create({});  
const ObjectWithData = Object.create({}, { someData: { data: 'data' } });  
  
const someObj = { someInfo: 'info' };  
const copyObject = Object.assign({}, someObj);
```

С помощью Object.assign() также можно копировать объекты.

```
const someObj = { someInfo: 'info' };  
const copyObject = Object.assign({}, someObj);  
  
console.log(copyObject); // { someInfo: 'info' }  
  
someObj.someInfo = 'new info';  
  
console.log(copyObject); // { someInfo: 'info' } объект не изменился!
```

Помимо этого метода, для копирования можно использовать методы: JSON.parse(), JSON.stringify(), а также библиотеки и самописные методы.

Как и в случае с копированием, сравнение объектов обычными способами с использованием ==, === и даже Object.is() не даст нам должных результатов.

```
const obj = {  
  name: 'Gayan Rosling',  
};  
  
const anotherObj = {  
  name: 'Gayan Rosling',  
};  
  
console.log(obj === anotherObj) // false, ссылки не равны  
console.log(Object.is(obj, anotherObj)) // false
```

Существует несколько способов сравнения объектов.

- **Ручное сравнение** — создание функции, которая принимает на вход два объекта одинакового типа и сравнивает их по конкретным полям;
- **Поверхностное сравнение** — создание функции, которая принимает на вход два объекта и сравнивает их на основе свойств верхнего уровня;
- **Глубокое сравнение** — создание функции, которая принимает на вход два объекта и сравнивает их на основе свойств на всех доступных уровнях вложенности.

Полезные методы объектов:

- **Object.keys(obj)** - метод, который возвращает массив строковых ключей объекта obj.
- **Object.values(obj)** - метод, который возвращает массив значений объекта obj.
- **Object.entries(obj)** - метод, который возвращает массив массивов, каждый из которых содержит пару ключ-значение объекта obj.
- **Object.assign(target, ...sources)** - метод, который копирует свойства всех исходных объектов sources в целевой объект target. Если свойство уже существует в целевом объекте, то оно будет перезаписано.
- **Object.create(proto, [propertiesObject])** - метод, который создает новый объект с указанным прототипом proto. Если передан второй аргумент propertiesObject, то он используется для определения свойств нового объекта.
- **obj.hasOwnProperty(prop)** - метод, который возвращает true, если объект obj содержит свойство с именем prop, иначе возвращает false. Этот метод не проверяет наличие свойства в прототипе объекта.
- **obj.toString()** - метод, который возвращает строковое представление объекта obj. По умолчанию, этот метод возвращает строку "[object Object]".

Массивы

Массив, как и объект, представляет данные в виде некоторого набора пар вида "ключ: значение". В отличие от объектов, данные в массиве располагаются в виде проиндексированного списка.

С помощью индексов можно очень просто обращаться к значениям массива. Для этого нужно просто указать нужный индекс в квадратных скобках сразу после имени соответствующего массива (например, fruits[2]).

Как и объекты, массивы имеют несколько способов создания:

```
const emptyArr = [];  
const arr = [1, 2, null, 'someStr'];  
const newArr = new Array();  
const arrayWithEmptyElems = new Array(5); // [empty, empty, empty, empty, empty]
```

Полезные методы массивов:

- **arr.length** - свойство, которое возвращает количество элементов в массиве arr.

- **arr.push(element1, ..., elementN)** - метод, который добавляет один или несколько элементов в конец массива arr и возвращает новую длину массива.
- **arr.pop()** - метод, который удаляет последний элемент из массива arr и возвращает его значение.
- **arr.unshift(element1, ..., elementN)** - метод, который добавляет один или несколько элементов в начало массива arr и возвращает новую длину массива.
- **arr.shift()** - метод, который удаляет первый элемент из массива arr и возвращает его значение.
- **arr.slice(start, end)** - метод, который создает новый массив, содержащий элементы от индекса start до индекса end (не включая его) из массива arr. Если аргумент end не указан, то будут выбраны все элементы от индекса start до конца массива.
- **arr.splice(start, deleteCount, item1, item2, ...)** - метод, который изменяет содержимое массива arr, удаляя элементы, начиная с индекса start и заменяя их на элементы item1, item2, и т.д. Если аргумент deleteCount не указан, то будут удалены все элементы от индекса start до конца массива.
- **arr.concat(arr1, arr2, ...)** - метод, который создает новый массив, содержащий элементы из массива arr, а также элементы из массивов arr1, arr2, и т.д.
- **arr.join(separator)** - метод, который создает строку, содержащую все элементы массива arr, разделенные указанным разделителем separator. По умолчанию, разделителем является запятая.
- **arr.indexOf(searchElement, fromIndex)** - метод, который возвращает индекс первого вхождения элемента searchElement в массиве arr, начиная с индекса fromIndex. Если элемент не найден, то метод возвращает -1.
- **arr.lastIndexOf(searchElement, fromIndex)** - метод, который возвращает индекс последнего вхождения элемента searchElement в массиве arr, начиная с индекса fromIndex. Если элемент не найден, то метод возвращает -1.
- **arr.includes(searchElement, fromIndex)** - метод, который возвращает true, если элемент searchElement присутствует в массиве arr, начиная с индекса fromIndex, иначе возвращает false.
- **arr.reverse()** - метод, который изменяет порядок элементов массива arr на обратный.
- **arr.sort([compareFunction])** - метод, который сортирует элементы массива arr в порядке возрастания или убывания. Если аргумент compareFunction не указан, то элементы сортируются в лексикографическом порядке.
- **arr.filter(callback[, thisArg])** - метод, который создает новый массив, содержащий элементы массива arr, для которых функция callback возвращает true.
- **arr.map(callback[, thisArg])** - метод, который создает новый массив, содержащий результаты вызова функции callback для каждого элемента массива arr.
- **arr.reduce(callback[, initialValue])** - метод, который применяет функцию callback к каждому элементу массива arr, накапливая результаты в переменной accumulator. Если аргумент initialValue указан, то он используется в качестве начального значения переменной accumulator.
- **arr.forEach(callback[, thisArg])** - метод, который вызывает функцию callback для каждого элемента массива arr, не создавая новый массив.

Циклы

Итерируемые сущности — это такие сущности, по которым возможно осуществить проход (или перебрать). Такая сущность представляет собой набор элементов, которые могут быть перебраны в цикле `for...of`. К ним относятся массивы, строки, наборы, карты и другие объекты, которые реализуют интерфейс итератора. Не все объекты могут быть перебраны в цикле `for...of`, так как могут не являться итерируемыми. Для перебора свойств таких объектов нужно использовать цикл `for...in`.

```
const arr = [1, 2, 3];

for (let elem of arr) {
  console.log(elem); // выведет 1, 2, 3
}

const obj = { a: 1, b: 2, c: 3 };

for (let prop in obj) {
  console.log(prop); // выведет 'a', 'b', 'c'
}
```

Функции

В JavaScript существует три основных типа функций:

- Объявляемая функция (function declaration);

Объявляемые функции — это функции, которые объявляются самым обычным способом с помощью ключевого слова function.

Такие функции имеют одну довольно значимую особенность - они «всплывают». Это означает, что такие функции могут быть использованы (вызваны) раньше, чем фактически объявлены в коде.

```
sayHi('John'); // Hi, John!

function sayHi(name) {
  console.log(`Hi, ${name}!`);
}
```

- Функциональное выражение (function expression);

Функциональные выражения — это способ объявления функций через переменную. Т.е. мы описываем функцию обычным способом, но вместо задания имени складываем её в переменную.

В таком случае «всплывать» функция уже не будет. Обратиться к ней можно будет лишь после непосредственной инициализации.

```
sayHi('Katua'); // ReferenceError: Cannot access 'sayHi' before initialization

const sayHi = function (name) {
  console.log(`Hi, ${name}!`);
};
```

- Стрелочная функция (arrow function, ES6).

```
sayHi('Katua'); // ReferenceError: Cannot access 'sayHi' before initialization

const sayHi = (name) => {
  console.log(`Hi, ${name}!`);
};

sayHi('Katua'); // Hi, Katua!
```

И ещё один специфичный тип:

- Именованное функциональное выражение (named function expression, NFE).

Кроме этого есть ещё некоторые «состояния» функций:

- Анонимная функция (anonymous function);
- Callback-функция (callback function);
- Функция-конструктор (constructor function);
- Функция высшего порядка (higher-order function).

В качестве аргумента функции может быть всё, что угодно. Можно передать туда примитив, объект, функцию или массив. Если в качестве аргумента передается объект, то можно применить деструктуризацию:

```
const obj = {
  name: 'Rayan',
  surname: 'Gosling',
};

function foo(obj) {
  console.log(obj.name); // Rayan
  console.log(obj.surname); // Gosling

  // или

  const { name, surname } = obj;
  console.log(name); // Rayan
  console.log(surname); // Gosling
}

function foo2({ name, surname }) {
  console.log(name); // Rayan
  console.log(surname); // Gosling
}

foo(obj);
foo2(obj);
```

Асинхронность. Event loop

Язык JavaScript – синхронный, с возможностью выполнения асинхронных задач. Эту возможность предоставляет такой механизм, как Event Loop.

Event Loop (цикл событий) — это механизм в JavaScript, который позволяет управлять выполнением асинхронного кода. Он отвечает за обработку событий, вызов колбэков и управление порядком выполнения задач.

В рамках Event Loop выделяются три типа задач:

1. Синхронные задачи (Synchronous tasks) — это задачи, которые выполняются синхронно, то есть поочередно, блокируя выполнение других задач до завершения. Например, выполнение простых выражений или операций.
2. Микрозадачи (Microtasks) — это задачи, которые имеют более высокий приоритет и должны быть выполнены сразу после завершения текущей операции. Обычно это промисы (Promises) и колбэки из методов Promise, MutationObserver и process.nextTick.

```
fetch('/api/v1/some/endpoint').then((res) => { // запрос на сервер. Промис ->
  микротаска
  return res.json()
}).catch((err) => {
  throw new Error(err)
})
```

```
})
```

```
let url = '/api/v1/some/endpoint';
let response = await fetch(url); // то же самое, только через async/await ->
микрозадача
let result = await response.json();
```

3. Макрозадачи (Macrotasks) — это задачи, которые имеют более низкий приоритет и выполняются после микрозадач. К ним относятся таймеры (setTimeout, setInterval), I/O операции, а также события ввода/вывода.

Порядок выполнения задач в Event Loop следующий:

1. Выполняются все синхронные задачи, которые находятся в стеке вызовов (вызов функций, console.log() и т.д.).
2. После завершения синхронных задач проверяются микрозадачи. Если есть микрозадачи, они выполняются до тех пор, пока очередь микрозадач не опустеет.
3. Затем выполняются макрозадачи: проверяются таймеры, I/O операции и другие асинхронные события.
4. Процесс повторяется: после завершения макрозадач снова проверяются микрозадачи, а затем продолжается выполнение макрозадач.

Взаимодействие с DOM

Язык JavaScript изначально был создан для веб-браузеров. Сегодня JavaScript может использоваться в браузере, на веб-сервере или в какой-то другой среде. Каждая среда предоставляет свою функциональность, которую спецификация JavaScript называет *окружением*. Окружение предоставляет свои объекты и дополнительные функции, в дополнение базовым языковым. Браузеры, например, дают средства для управления веб-страницами, такие как объекты navigator, screen, document или функции setTimeout, setInterval, prompt, confirm и т.д.

Document Object Model, сокращённо DOM – объектная модель документа, которая представляет все содержимое страницы в виде объектов, которые можно менять.

Объект document – основная «входная точка». С его помощью мы можем что-то создавать или менять на странице.

Для работы с HTML документов нужно в файле index.html подключить скрипт внутри <head>:

```
<head>
...
<script src="./index.js" async></script>
</head>
```

Поменяем фон у блока с id = 'practice':

```
const practiceDiv = document.getElementById('practice');
practiceDiv.style.background = 'blue';
```



PRACTICE

ABOUT

SKILLS

PORTFOLIO

CONTACTS

PRACTICE

Заголовки

Заголовки на сайте могут быть разных уровней, начиная от заголовка первого уровня (H1) и заканчивая заголовками шестого уровня (H6). Использование заголовков является семантически верным, так как позволяет структурировать содержимое страницы и указать наиболее важные разделы.

Браузеры и поисковые системы используют семантическую верстку для более эффективного индексирования страниц и повышения их релевантности для пользователей. Кроме того, использование заголовков упрощает доступ к информации для людей с ограниченными возможностями, таких как слабовидящие или незрячие пользователи, которые могут использовать программы чтения экрана для навигации по сайту.

Заголовок первого уровня

Заголовок второго уровня

Заголовок третьего уровня

Заголовок четвертого уровня

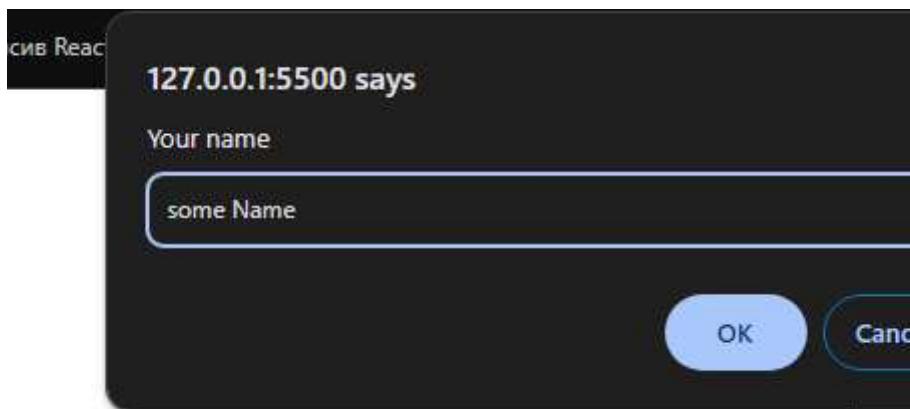
Заголовок пятого уровня

Заголовок шестого уровня

Фон изменил цвет.

Попробуем получить данные с нашей страницы:

```
const personName = prompt('Your name');  
console.log(personName);
```



some N

В реальной жизни методы `prompt` и `alert` не используются, но знать это будет не лишним.

Попробуем поменять цвета у заголовка с помощью JS:

```
<h1 class="title" id="titleName">Practice</h1>
```

```
const title = document.getElementById('titleName'); // получаем заголовок
title.style.transition = 'all 0.3s ease'; // задаем свойство для плавности
const colors = ['#c47068', '#c4b068', '#68c485', '#68b8c4', '#687ac4', '#a468c4']; //
массив цветов

setInterval(() => {
  title.style.color = `${colors[Math.floor(Math.random() * colors.length)]}`; // рандомно
  получаем цвет и задаем стиль
}, 500); // устанавливаем интервал в 500 мс
```

PRACTICE

PRACTICE

PRACTICE

События

Событие — это сигнал от браузера о том, что что-то произошло.

Список самых часто используемых DOM-событий:

События мыши:

- **click** – происходит, когда кликнули на элемент левой кнопкой мыши (на устройствах с сенсорными экранами оно происходит при касании).
- **contextmenu** – происходит, когда кликнули на элемент правой кнопкой мыши.
- **mouseover** / **mouseout** – когда мышь наводится на / покидает элемент.
- **mousedown** / **mouseup** – когда нажали / отжали кнопку мыши на элементе.
- **mousemove** – при движении мыши.

События на элементах управления:

- **submit** – пользователь отправил форму `<form>`.
- **focus** – пользователь фокусируется на элементе, например нажимает на `<input>`.

Клавиатурные события:

- **keydown** и **keyup** – когда пользователь нажимает / отпускает клавишу.

События документа:

- **DOMContentLoaded** – когда HTML загружен и обработан, DOM документа полностью построен и доступен.

CSS events:

- **transitionend** – когда CSS-анимация завершена.

Событию можно назначить обработчик, то есть функцию, которая сработает, как только событие произошло. Именно благодаря обработчикам JavaScript-код может реагировать на действия пользователя. Есть несколько способов назначить событию обработчик:

- Использование атрибута HTML

Обработчик может быть назначен прямо в разметке, в атрибуте, который называется `on<событие>`.

Можно задать функцию прямо внутри

```
<h1 class="title" id="titleName" onClick="alert('hi')">Practice</h1>
```

либо же указать название функции из файла `index.js`

```
<h1 class="title" id="titleName" onClick="startChangingColor()">Practice</h1>
```

```
function startChangingColor() {
  setInterval(() => {
    title.style.color = `${colors[Math.floor(Math.random() * colors.length)]}`;
  }, 500);
}
```

- Использование свойства DOM-объекта

Можно назначить обработчик, используя свойство DOM-элемента `on<событие>`:

```
<h1 class="title" id="titleName" onClick="startChangingColor()">Practice</h1>
<script>
  titleName.style.transition = 'all 0.3s ease';
  const colors = ['#c47068', '#c4b068', '#68c485', '#68b8c4', '#687ac4', '#a468c4'];

  titleName.onclick = function startChangingColor() {
    setInterval(() => {
      titleName.style.color = `${colors[Math.floor(Math.random() * colors.length)]}`;
    }, 500);
  };
</script>
```

Так как у элемента DOM может быть только одно свойство с именем `onclick`, то назначить более одного обработчика так нельзя.

- **AddEventListener**

Фундаментальный недостаток описанных выше способов назначения обработчика – невозможность повесить несколько обработчиков на одно событие, поэтому были добавлены специальные методы `addEventListener` и `removeEventListener`.

Для удаления нужно передать именно ту функцию-обработчик которая была назначена. Если функцию обработчик не сохранить где-либо, мы не сможем её удалить. Нет метода, который позволяет получить из элемента обработчики событий, назначенные через `addEventListener`.

```
function handler() {
  console.log('Спасибо!');
}
```

```
input.addEventListener("click", handler);
```

```
input.removeEventListener("click", handler);
```

Метод `addEventListener` позволяет добавлять несколько обработчиков на одно событие одного элемента, например:

```
<input id="elem" type="button" value="Нажми меня"/>

<script>
  function handler1() {
    console.log('Спасибо!');
  };

  function handler2() {
    console.log('Спасибо ещё раз!');
  }

  elem.onclick = () => alert("Привет");
  elem.addEventListener("click", handler1); // Спасибо!
  elem.addEventListener("click", handler2); // Спасибо ещё раз!
</script>
```

Чтобы хорошо обработать событие, могут понадобиться детали того, что произошло. Не просто «клик» или «нажатие клавиши», а также – какие координаты указателя мыши, какая клавиша нажата и так далее.

Когда происходит событие, браузер создаёт объект события, записывает в него детали и передаёт его в качестве аргумента функции-обработчику.

Пример ниже демонстрирует получение координат мыши из объекта события:

```
<input type="button" value="Нажми меня" id="elem">

<script>
  elem.onclick = function(event) {
    // вывести тип события, элемент и координаты клика
    alert(event.type + " на " + event.currentTarget);
    alert("Координаты: " + event.clientX + ":" + event.clientY);
  };
</script>
```

В реальной разработке чаще всего встречается пример получения данных интерактивных объектов:

```
<input type="button" value="Нажми меня" id="elem">

<script>
  elem.onchange = function(event) {
    const inputData = event.target.value;
    console.log(inputData);
  };
</script>
```

Выполнение работы

1 В созданном ранее проекте ([П72 HTML \(Hypertext Markup Language\), CSS \(Cascading Style Sheets\)](#)) создайте файл `index.js`.



2 Подключите файл index.js к index.html. Для этого в теге <head> файла index.html добавьте

```
<script src="./index.js" async></script>
```

Валидация формы

3 В файле index.html добавьте атрибуты id и name ко всем интерактивным полям, а также атрибут disabled="true" элементу button:

```
<input name="text" type="text" id="text" placeholder="Любой текст" />
<input name="number" id="number" type="number" placeholder="Только числа"/>
<input name="phone" type="tel" placeholder="Номер телефона" id="phone"/>
<input name="email" type="email" id="email" placeholder="Почта" />
<input name="password" type="password" id="password" placeholder="Пароль"/>
...
<button type="submit" id="formSubmitBtn" disabled="true">Отправить</button>
```

Это необходимо для корректной обработки элементов формы скриптом.

4 В файле style.css добавьте стили для кнопки:

```
.practice button:disabled {
  background: gray;
  border: 1px solid gray;
  cursor: auto;
}

.practice button:disabled:hover {
  background: gray;
  border: 1px solid gray;
}
```

Результат:



Кнопка "Отправить" стала серой, при этом на нее нельзя нажать.

5 В файле index.js объявите переменные:

```
const passwordInput = document.getElementsByName('password')[0];
const submitButton = document.getElementById('formSubmitBtn');
const emailInput = document.getElementById('email');
```

В первой строке объявлена переменная passwordInput, которая будет отвечать за взаимодействие с полем ввода пароля.

document – обращение к DOM

document.getElementsByName('password')[0] – функция, позволяющая использовать в скрипте именно то поле, name которого password. Однако, метод getElementsByName возвращает не единственное значение, а массив элементов с заданным name. В нашем случае такое поле всего одно, поэтому получаем его через индекс – [0]. Name – не уникальный атрибут;

document.getElementById('formSubmitBtn') – функция, позволяющая использовать в скрипте именно то поле, id которого formSubmitBtn. ID – уникальный атрибут. Данный способ обычно предпочтительнее.

6 Объявите функцию для изменения стиля полей:

```
function validationStyle(condition, element) {
  if (condition) {
    element.style.borderColor = 'green';
  } else {
    element.style.borderColor = 'red';
  }
}
```

Данная функция изменяет контур поля ввода по условию: зеленый, если condition истинно, красный – если ложно.

7 Объявите функции обработки полей ввода:

```
function passwordValidate(password) {
  const regex = /^(?=.*[0-9])(?=.*[!@#$%^&*])(?=.*[a-z])(?=.*[A-Z])[0-9a-zA-Z!@#$%^&*]{6,}/;
  validationStyle(regex.test(password.value), password);
  return regex.test(password.value);
}
```

```
function emailValidate(email) {
  const regex = /^[a-zA-Z0-9._-]+@[a-zA-Z0-9._-]+\.[a-zA-Z0-9_-]+$/;
  validationStyle(regex.test(email.value), email);
  return regex.test(email.value);
}
```

Переменная regex с блочной областью видимости – [регулярное выражение](#). Веб-ресурсы для помощи в разработке регулярных выражений ([regexr.com](#), [regex101.com](#), [ihateregex.io](#))

Функция возвращает true или false – совпадает ли значение поля с регулярным выражением regex, а также вызывает функцию для изменения стиля поля ввода.

8 Объявите функцию для изменения атрибута "disabled" у кнопки "Отправить":

```
function checkInputs() {
  const isEmailValid = emailValidate(emailInput);
  const isPasswordValid = passwordValidate(passwordInput);
  if (isEmailValid && isPasswordValid) {
    submitBtn.removeAttribute('disabled');
  } else {
    submitBtn.setAttribute('disabled', 'true');
  }
}
```

Данная функция удаляет атрибут disabled у кнопки "Отправить", если пройдена проверка для полей "Почта" и "Пароль".

9 Добавьте в index.html после поле ввода пароля элемент checkbox с надписью:

```
<input type="checkbox" id="showPassword" name="showPassword" />
<label for="showPassword">Показать пароль</label>
```

10 Объявите переменную и соответствующую функцию для обработки события по смене состояния checkbox:

```
const showPasswordBtn = document.getElementById('showPassword');
function showPassword() {
  if (passwordInput.type === 'password') {
    passwordInput.type = 'text';
  } else {
    passwordInput.type = 'password';
  }
}
```

```
}
}
```

Данная функция изменяет атрибут type поля ввода password при изменении значения checkbox.

11 Объявите слушателей событий:

```
passwordInput.addEventListener('change', checkInputs);
emailInput.addEventListener('change', checkInputs);
showPasswordBtn.addEventListener('click', showPassword);
```

Данные методы вызывают функцию checkInputs для passwordInput или emailInput, объявленную ранее при изменении содержимого соответствующего поля.

Для checkbox данный методы вызывает функци. showPassword при событии click, то есть по щелчку мыши.

Работа с медиа-устройствами

Камера

В качестве примера рассмотрим работу с камерой ([по документации Mozilla](#)).

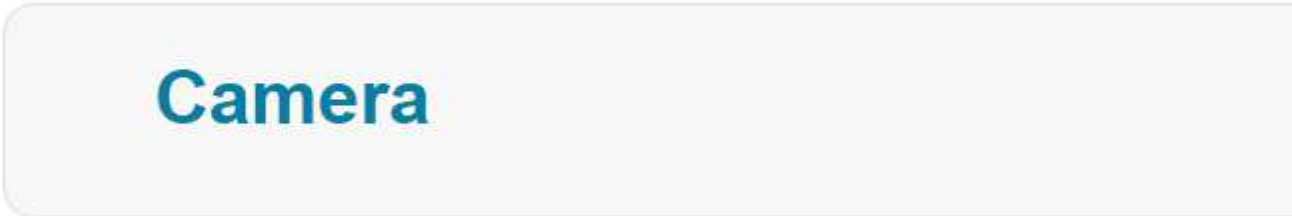
12 В текущем проекте создайте новый файл capture.js и подключите его в head index.html:

```
<script src="capture.js" defer></script>
```

13 В index.html добавьте в div класса practice еще один div класса block camera и добавьте заголовок второго уровня:

```
<div class="block camera">
  <h2>Camera</h2>
</div>
```

Результат:



14 Внутри блока camera добавьте ползунки:

- width
- brightness
- opacity
- contrast
- saturate

Пример:

```
<input
  type="range"
  id="range_width"
  min="100"
  max="1000"
  step="100"
  value="320"
/>
<label for="range_width">width</label>
```

15 Добавьте тег "video":

```
<video id="video">Video stream not available.</video>
```

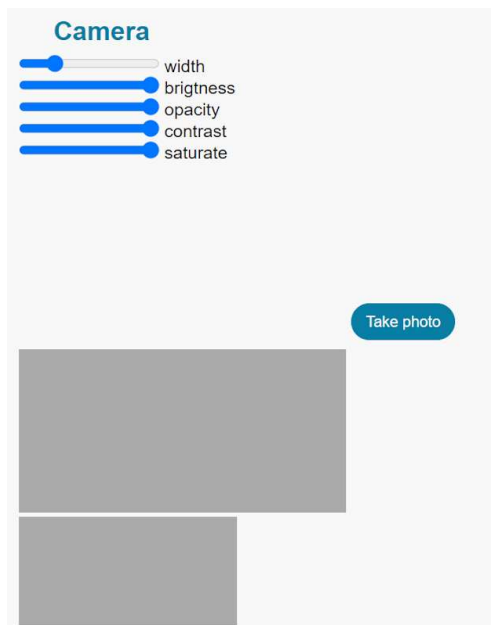
16 Добавьте кнопку Take photo:

```
<button id="startbutton">Take photo</button>
```

17 Добавьте тег "canvas":

```
<canvas id="canvas" class="output"> </canvas>
```

Результат:



18 По умолчанию элемент canvas, который будет сохранять захваченный кадр, показывается, поэтому его отображение можно отключить в стилевом файле:

```
#canvas {
  display: none
}
```

19 В созданном ранее файле capture.js объявите самовызывающуюся функцию и несколько переменных:

```
(function () {

  var width = 320;
  var height = 0;

  var streaming = false;

  var video = null;
  var canvas = null;
  var photo = null;
  var startbutton = null;
```

width Какой бы не был размер входящего видео, мы намерены масштабировать результирующее изображение к ширине в 320 px.

height Результирующая высота изображения будет вычисляться на основе переданной ширины и соотношению сторон потока с камеры.

streaming Указывает на текущую активность видеопотока.

video Будет содержать ссылку на элемент <video> после загрузки страницы.

canvas Содержит ссылку на элемент <canvas> после загрузки страницы.

photo Содержит ссылку на элемент после загрузки страницы.

startbutton Содержит ссылку на элемент `<button>` после загрузки страницы, используется для старта захвата.

20 Объявите метод, который запрашивает у пользователя доступ к камере, инициализирует элемент `<img>` в значение по умолчанию, устанавливает обработчики событий и запускает захват изображения при нажатии на кнопку.

```
function startup() {
```

Ссылки на основные элементы:

```
video = document.getElementById('video');
canvas = document.getElementById('canvas');
photo = document.getElementById('photo');
startbutton = document.getElementById('startbutton');
```

Получение медиапотока:

```
navigator.mediaDevices.getUserMedia({ video: true, audio: false })
  .then(function (stream) {
    video.srcObject = stream;
    video.play();
  })
  .catch(function (err) {
    console.log('An error occurred: ' + err);
  });
```

Обработка события (canplay) начала воспроизведения:

```
video.addEventListener(
  'canplay',
  function (ev) {
    if (!streaming) {
      height = video.videoHeight / (video.videoWidth / width);

      if (isNaN(height)) {
        height = width / (4 / 3);
      }

      video.setAttribute('width', width);
      video.setAttribute('height', height);
      canvas.setAttribute('width', width);
      canvas.setAttribute('height', height);
      streaming = true;
    }
  },
  false,
);
```

Обработка нажатий кнопки и завершение метода:

```
startbutton.addEventListener(
  'click',
  function (ev) {
    takepicture();
    ev.preventDefault();
  },
  false,
);

clearphoto();
}
```

21 Объявите метод создания изображения и преобразования его в формат :

```
function clearphoto() {  
  var context = canvas.getContext('2d');  
  context.fillStyle = '#AAA';  
  context.fillRect(0, 0, canvas.width, canvas.height);  
  
  var data = canvas.toDataURL('image/png');  
  photo.setAttribute('src', data);  
}
```

22 Объявите метод для захвата кадра из видеопотока:

```
function takepicture() {  
  var context = canvas.getContext('2d');  
  if (width && height) {  
    canvas.width = width;  
    canvas.height = height;  
    context.drawImage(video, 0, 0, width, height);  
  
    var data = canvas.toDataURL('image/png');  
    photo.setAttribute('src', data);  
  } else {  
    clearphoto();  
  }  
}
```

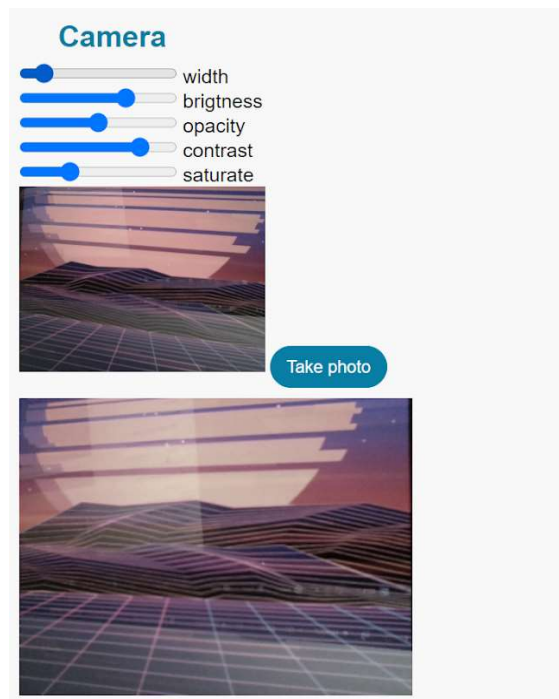
23 Опишите обработку ползунков. Пример:

```
const currentWidth = document.getElementById('range_width');  
currentWidth.addEventListener('change', () => {  
  width = currentWidth.value;  
  height = video.videoHeight / (video.videoWidth / width);  
  video.width = currentWidth.value;  
  video.height = height;  
});  
  
const brightness = document.getElementById('range_brightness');  
brightness.addEventListener('change', () => {  
  video.style.filter = `brightness(${brightness.value / 100})`;  
  canvas.style.filter = `brightness(${brightness.value / 100})`;  
});
```

24 Завершите функцию function вызовом слушателя:

```
window.addEventListener('load', startup, false);  
})();
```

Результат:



Содержание отчета

- pdf-версия разработанной страницы (файл – печать – pdf). Настройки печати:

Страницы	Все
Ориентация	Горизонтальная
Цветная печать	Цветная печать
Дополнительные настройки	
Размер бумаги	A4
Число страниц на листе	1
Поля	Минимальные
Масштаб	Персонализированные
	80
Параметры	☒ Верхние и нижние колонтитулы ☒ Фон

- Листинги html, css-файлов
- Листинги js-файлов с комментариями
- *Приложить к отчету файлы проекта (по требованию преподавателя, по умолчанию требуется).

Самостоятельная работа

- 1 Реализуйте стили для checkbox "Показать пароль".
- 2 Реализуйте стили для блока "Camera".
- 3 "Оживите" форму, разработанную в S3W02: сделайте так, чтобы кнопка "Отправить" давала какой-то ответ, например, перенаправление на другую страницу, которая сообщает "Спасибо за заполнение формы". Все кнопки и ссылки

должны быть рабочими, в т. ч. навигационное меню. Реализуйте плавную прокрутку.

[ШАБЛОН ОТЧЕТА]